Agile Software Development – Scrum Method

Software/System Architecture

Dr. Basem Suleiman
School of Computer Science and Engineering

Agenda

- Agile Software Development Model
- Scrum Method
  - Agile Values & Principles
  - Teams and Roles, Events, Artifacts
  - Estimation and Progress Monitoring
- Agile software management tools
  - Jira
- Software/System Architecture

Agile Development Model

Agile Development – Class Interaction

- Describe your understanding of Agile software development using key words? If not experienced with Agile, type "Not Familiar"

Slido.com --> # 3301735

Software Project Failure – the trigger for Agility

- Software is inherently **flexible** and can **change**
  - **Requirements change** through changing business circumstances; the software that supports it must also **evolve** and change
- Project failure - **extended time to develop a system**
- **Costs** escalated and r**equirements' changes**
- Agile methods intend to **develop systems more quickly with limited time spent on analysis and design**

Agile Software Development Process Model

Agile methods are considered

- **Light-weight**
- **People-based** rather than Plan-based

Several agile methods

- No single agile method
- Extreme Programming (XP), Scrum, Lean

Agile Manifesto closest to a definition

- Set of principles
- Developed by Agile Alliance

Agile Manifesto – Agile Values

"We are uncovering **better ways of developing software** by doing it and helping others do it. Through this work we have come to **value**:"

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

The items on the left are **more valued** than those at the right

Agile Manifesto: http://agilemanifesto.org/

Agile Principles (1)

1. Our highest priority is to **satisfy the customer** through early and continuous delivery of valuable software

2. **Welcome changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage

3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the **shorter timescale**

4. **Business people** and **developers** must **work together daily** throughout the project

Agile Alliance: http://www.agilealliance.org

Agile Principles (2)

5. Build projects around **motivated individuals**. Give them the **environment and support** they need, and **trust** them to get the job done

6. The most efficient and effective method of **conveying information** to and within a development team is **face-to-face conversation**

7. **Working software** is the primary **measure of progress**

8. Agile processes promote **sustainable development**. The sponsors, developers, and users should be able to maintain a **constant pace** indefinitely

Agile Alliance: http://www.agilealliance.org

Agile Principles (3)

9. **Continuous attention** to technical excellence and good design enhances agility

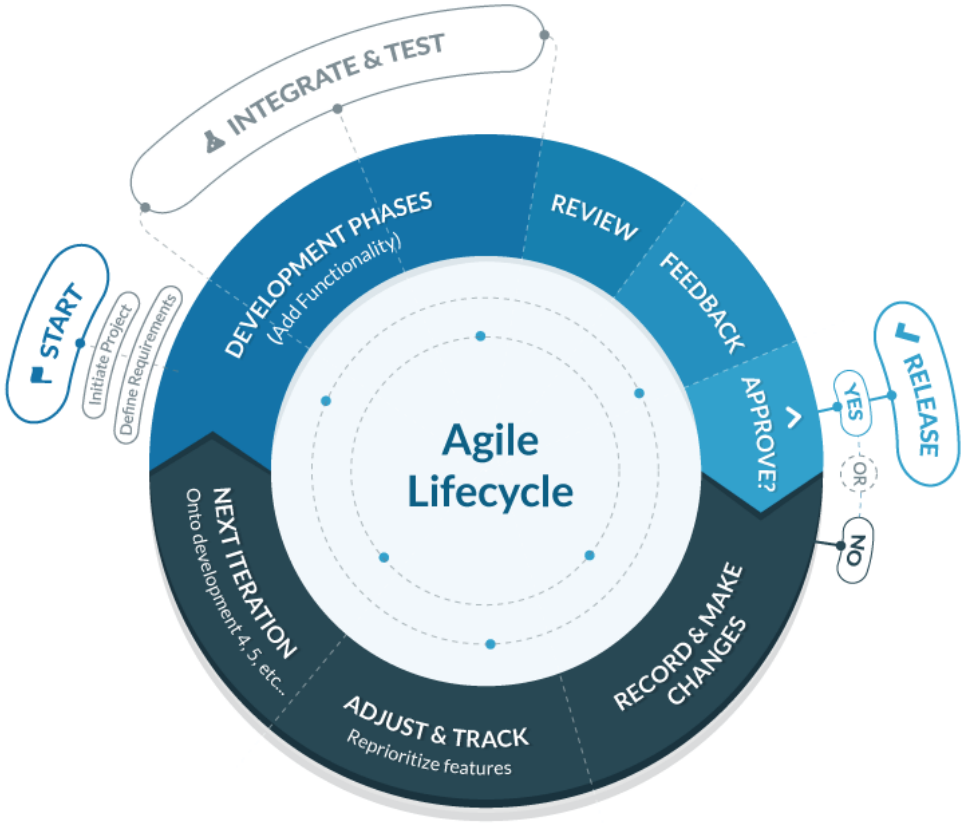
10. **Simplicity** – the art of maximizing the amount of work not done is essential

11. The best architectures, requirements, and designs emerge from **self-organizing teams**

12. At regular intervals, the **team reflects** on how to become more **effective**, then **tunes and adjusts its behavior** accordingly

Agile Alliance: http://www.agilealliance.org

Agile Software Development Model



Incremental & iterative development

https://blog.capterra.com/agile-vs-waterfall/

Scrum (Software Development)

- An agile method for **managing software development** focused on **delivering products of the highest value**
- Focus on **continuous improvement** of the **product**, the **team** and the **working environment**
- Scrum is lightweight, simple to understand but **difficult to master**

Scrum Practices

Teams and their **roles**
- Product Owner, Scrum Master, Dev Team

Events

•Sprint, Sprint Planning, Daily Scrum, Sprint Review, and Retrospective

Artifacts
- Product Backlog, Sprint Backlog, Increment
- Project estimation and Sprint estimation
- Rules govern the relationships between roles, events and artifacts
- All of the above components are essential for Scrum success

The Scrum Team

Scrum Team – The Product Owner

Product Owner maximize the value of the product and the work

- Talks to customer, understand requirements and its priorities
- Managing the Product Backlog (only person)
  - Can assign it to the development team, but still accountable

Managing the product backlog:

- Record product backlog items and order it
- Optimize the value of the work the development team performs
- Ensure transparency and clarity of the product backlog
- Ensure the development team understands product backlog

Scrum Team – The Development Team

- **Self-organizing**: turns Product Backlog into increments of potentially releasable functionality
- **Two-pizza team** (4-9 members)
- **Cross-functional**: skills mix necessary to create a product
- **No sub-teams**: regardless of domains that need to be addressed
- Whole team is **accountable**
- **No titles** for Dev team members

Scrum Team – The Scrum Master

Keeps the team focused on **using Scrum properly** (“*servant-leader*”)

- Everyone understands Scrum rules and values (coaching)
- Remove impediments
- Helps those outside the Scrum team which of their interactions with the Scrum team are/aren’t helpful
- Maximize the value created by the Scrum team through changing team interactions

Scrum Team – The Scrum Master

Serves the Product Owner;

- Creating mutual understanding of goals, scope and product domain
- Finding effective ways for managing the product backlog
- Helping the Scrum team understands the need for clear and concise product backlog items;
- Ensuring the product owner knows how to arrange the Product Backlog to maximize value;
- Understanding and practicing agility
- Facilitating Scrum events as requested or needed

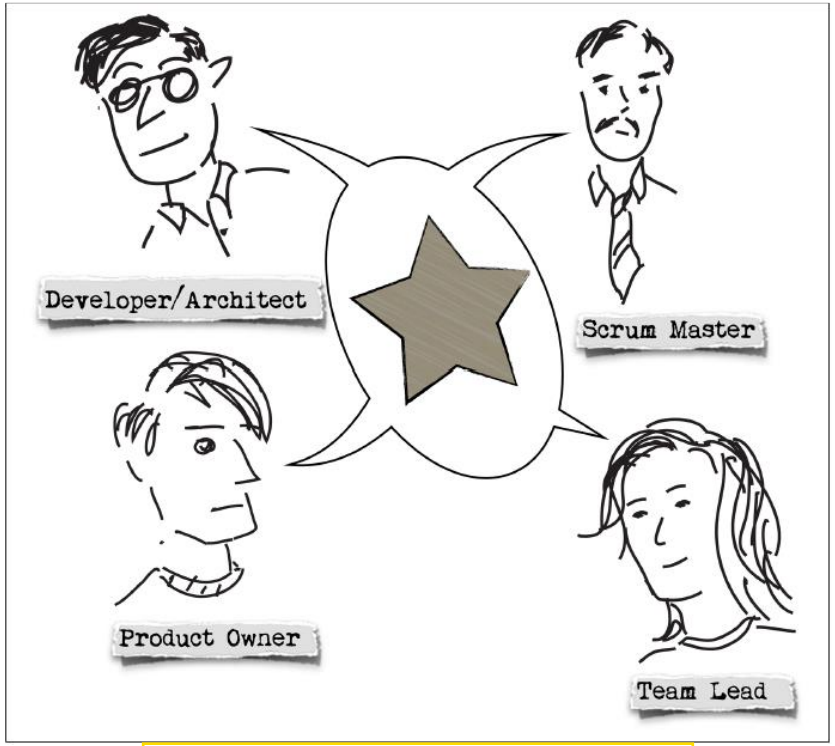
Scrum Team – The Scrum Master

Serves the Dev team

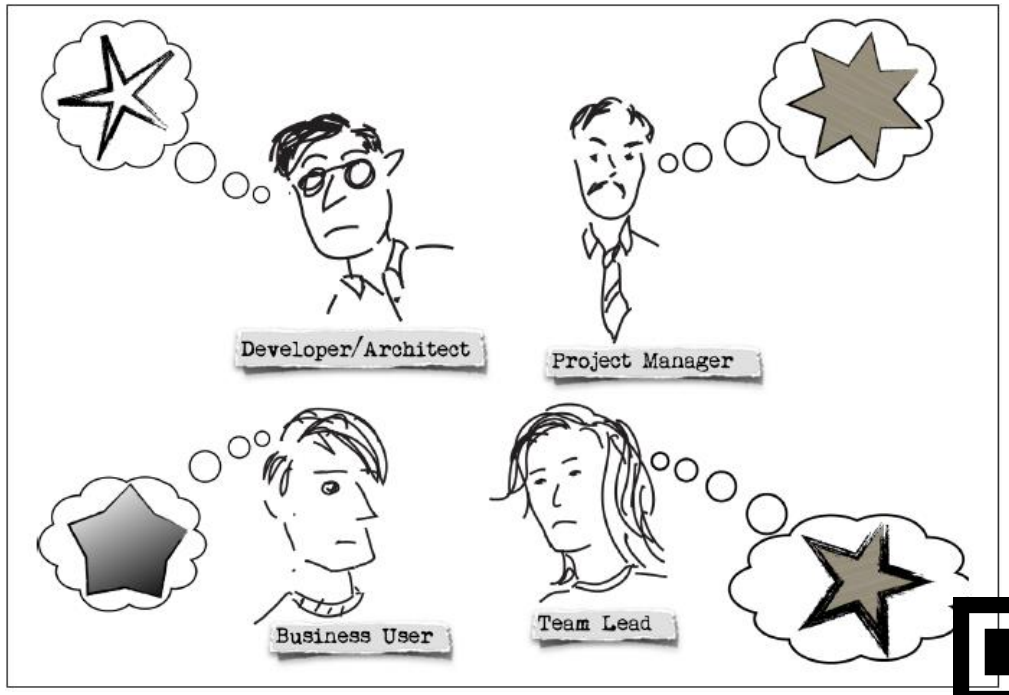
- To be self-organization and cross-functionality;
- To create high-value products;
- Removing impediments to the Dev team’s progress;
- Facilitating Scrum events as requested or needed; and,
- Coaching the Dev team to do Scrum properly

Scrum Team

Which team better describes the Scrum team structure?



Team A



Team B

Slido.com --> # 3301735

The Scrum Events

Scrum Events – The Sprint

A development **iteration** (one cycle)

- Useable and potentially **releasable product increment** is created

Time-boxed (typically 2-4 weeks)

- Too long sprints may lead to changes in the definition

Sprints have **consistent durations** during the product development

Consists of the **Sprint Planning, Daily Scrum, the Development Work, the Sprint Review and the Sprint Retrospective**

Scrum Events – Sprint Planning

Identify the **Sprint Goal** (items from the “Product Backlog”)

Identify **work to be done** to deliver this

Two-parts meeting (Scrum Master, Product Owner and Dev team)

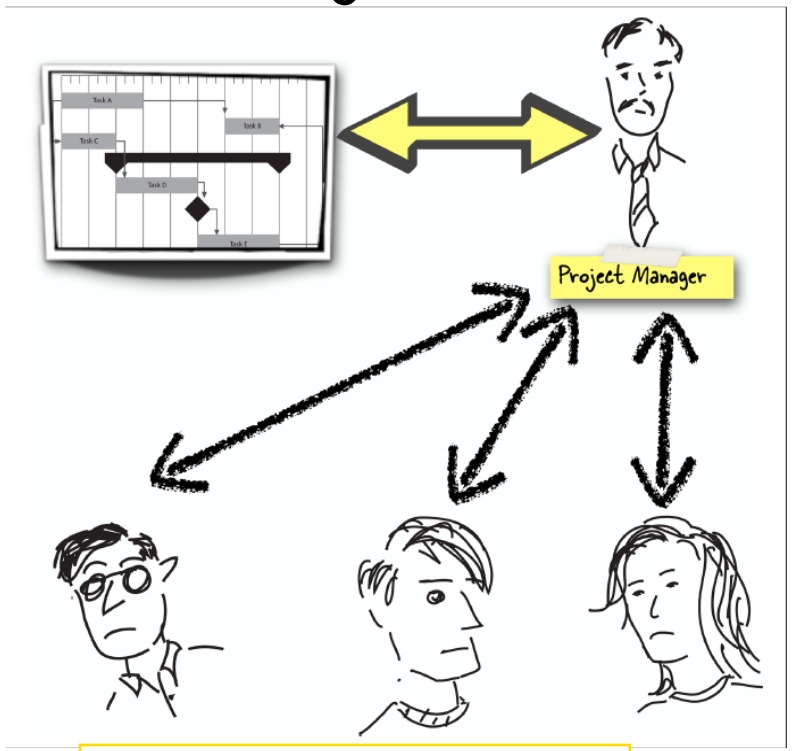
- **Before meeting**: Product Owner prepares prioritized list of most valuable items
- **Meeting part 1**: Product Owner & Dev team select items to be delivered at the end of the sprint based on their value and on the team’s estimate of how much work needed

- **Meeting part 2:** Dev team figure out the individual tasks they’ll use to implement those items

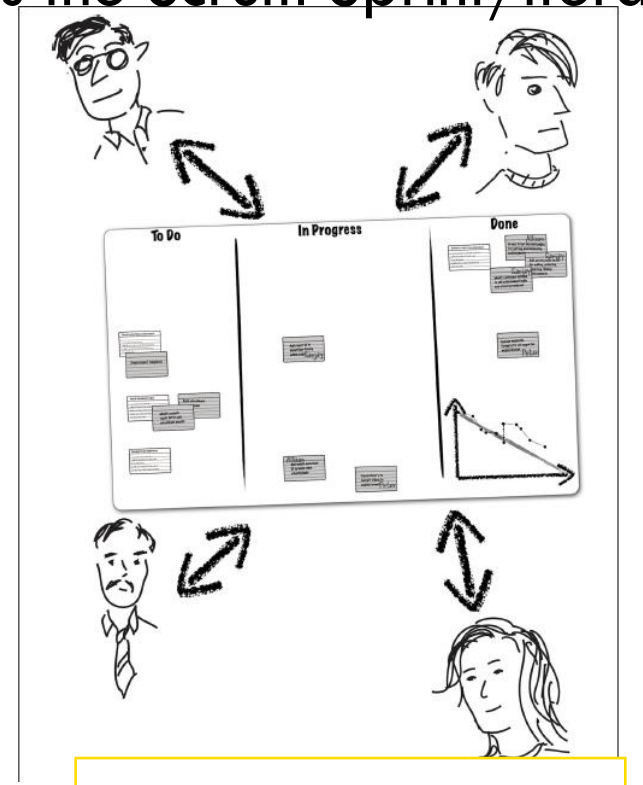
Output: Sprint Backlog

Scrum – Team Structure

Which team organization better describes the Scrum Sprint/iteration planning?



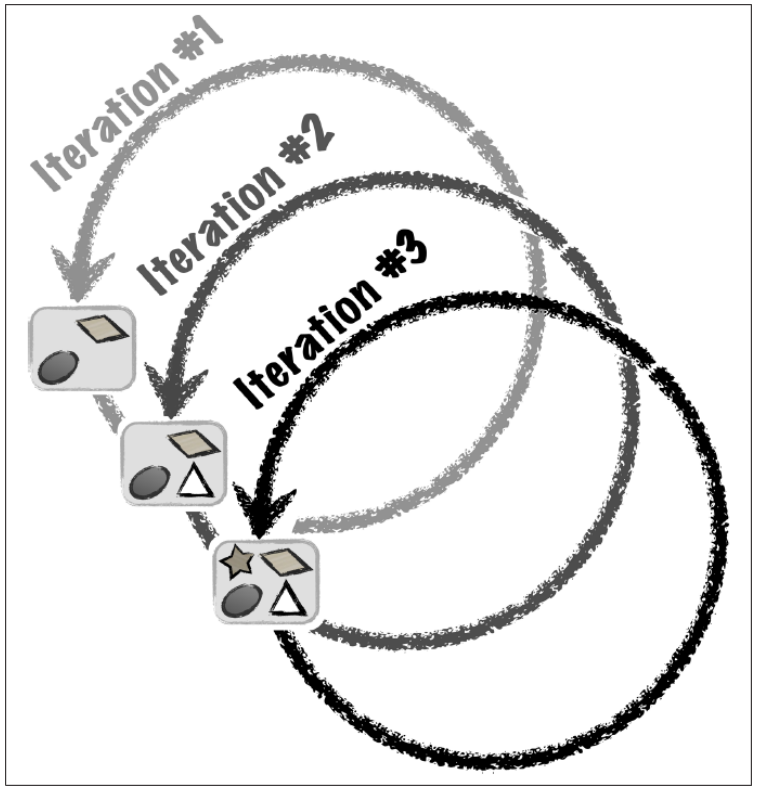
Team C

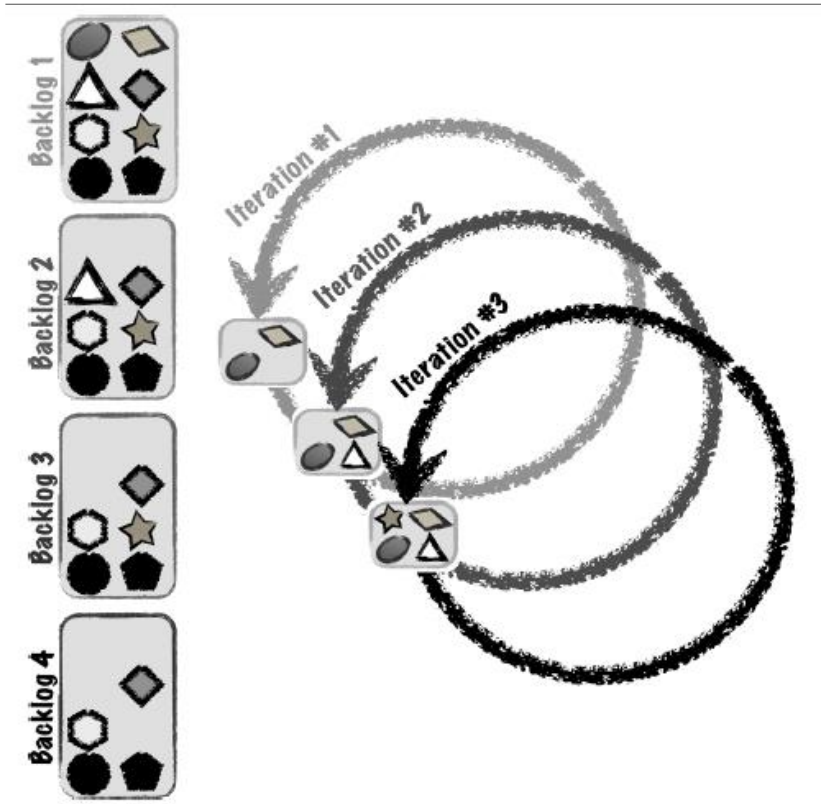


Team D

Slido.com --> # 3301735

Scrum Iteration Process





* Product Backlog: set of all features and sub-features (items) needed to build the product

Scrum Events – During Sprint

Daily Scrum meeting

- To ensure **problems and obstacles are visible** to the team
- **Timeboxed *15 minutes*** (same time and place each day)
- **All team members** including Scrum Master and Product Owner must attend

- Each briefly **answers three questions**:
  - What did I do yesterday that helped the development team meet the Sprint Goal?
  - What will I do today to help the development team meet the Sprint Goal?
  - do I see any obstacles that prevent me or the Dev team to meeting the Sprint Goal?

- **No problem-solving** during the meeting
  - Follow-up meetings if further discussion is required

Scrum Events – During The Sprint

Development Work; the Dev team

- Builds the items in the *Sprint Backlog* into working software
- Should inform the Product Owner if they are overcommitted or can add extra items if time allows
- Must **update the Sprint backlog** and keep it **visible** to everyone

Scrum Events – During The Sprint

The Sprint Review

Informal meeting at end of the Sprint (max. 4-hours)

- **Dev team demonstrates working software to customers/stakeholders**
  - Items done – completed, tested & accepted by the product owner
  - Only functional working software – not architecture, database design etc.

- **Stakeholders share** their feedback, ideas, feelings, thoughts about the demo

- The Product Owner explains “Done” and not “Done” items

Output: revised Product Backlog and probable items for next iteration

Scrum Events – Retrospectives

Opportunity for the **Scrum team to inspect itself and create plan for improvements**

Purpose:

- **Inspect** how the last **Sprint** went with regards to **people**, **relationships, process,** and **tools**;
- Identify and order the major **items that went well** and **potential improvements**;
- Create a plan for **implementing improvements** to the way the Scrum Team does its work

Scrum Events – Retrospectives

Retrospective meetings (max. 1-2 hours)

The Scrum Master and the Dev. team (maybe product owner)

Each person answer **two questions**:

- What went well during the Sprint?
- What can be improved in the future?

The **Scrum Master notes improvements** that should be added to the Product Backlog (non-functional items)

- E.g., set-up a better build server, adopting design principles, changing office layout

Output: **identified improvements** to be implemented in the next Sprint (adaptation)

Sprint Retrospective - Example for Sprint 1

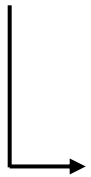
Last retrospective's To Try	***What went well***	***What didn't go so well***	***To Try***
First sprint so N/A	Met sprint goal	Duplicated work (2 team-members accidentally working on same task without knowing)	

Sprint Retrospective - Example for Sprint 1

Last retrospective's To Try	***What went well***	***What didn't go so well***	***To Try***
First sprint so N/A	Met sprint goal	Duplicated work (2 team-members accidentally working on same task without knowing)	ensure everyone sends daily scrum updates if absent from the daily scrum meeting Adam checks that everyone's present, and reminds any absentees to email status update to team)

Sprint Retrospective - Example for Sprint 2

Last retrospective's To Try	*What went well*	*What didn't go so well*	*To Try*
Ensure everyone sends daily scrum updates if absent from the daily scrum meeting. (Adam checks that everyone's present, and reminds any absentees to email status update to team)	Met sprint goal No more duplicated work	Software unit demonstrated was buggy	Write unit tests for each unit developed (all members to follow up on that)



This worked well. Everyone knows what everyone else is doing - no more duplicate work

Scrum Artifacts

Scrum Artifacts – Product Backlog

Set of **all features and sub-features** (items) needed to build the product (the "Plan" for multiple iterations)

- Features, functions, requirements, enhancements and fixes from previous Sprints

Maintained by the Product Owner in collaboration with customers and team

The source of the **product requirements**

- Evolves over the time and never complete (dynamic)

The **items ordered by priority** – value to the customer

- To deliver value to the customer in each iteration, put the most important things early

Scrum Artifacts – Sprint Backlog

Set of Product Backlog items selected for the Sprint, and a plan for delivering the product increment and realize the Sprint Goal

The Dev. team to forecasts next items to be implemented

Includes at least one high-priority improvement identified from previous Sprint

The Dev. team adds new work to the Sprint Backlog

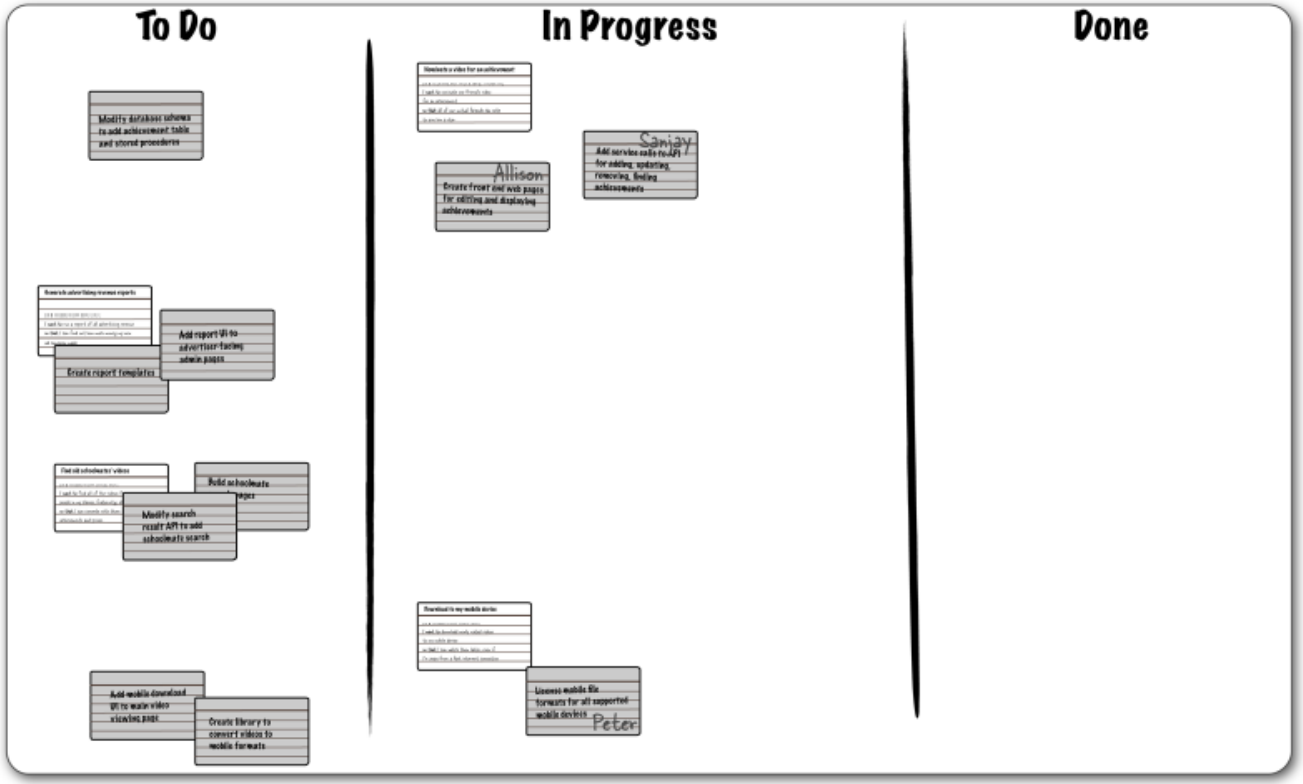
The estimated remaining work is updated once an item is completed

Visible to anyone and to be modified by the Dev. team

Sprint Backlog – Example

Typically divided into 3 sections;
To Do, In Progress, Done

Tools to support Sprint planning and monitoring; e.g., Jira



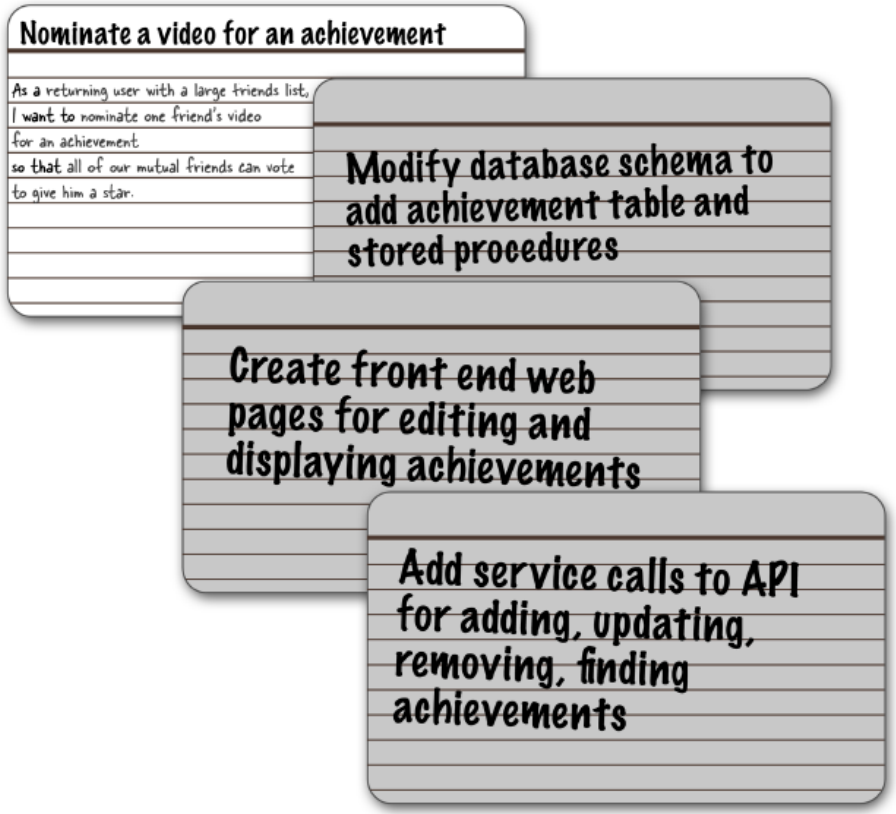
Agile Methods for Estimating Size

Scrum – Project and Iteration Estimation

- Product Owner works with the customer to set the priority of PB items
- In an iteration, the team divides the PB items into individual tasks
- The dev. team defines tasks and estimates the size/effort for each item
  - Product Owner/customers are not allowed to change the estimates
- The list of tasks is flexible
- Dev. team tracks all “tasks” on a Task Board (Backlog, in-progress, Done)
- Dev. team tracks progress with a burn-down chart

**PB* *Product Backlog*

User Stories – Base for Better Estimation



Estimation – Size of User Story

- **Ideal days**
  - The amount of time a user story will take to develop
  - Express the estimate as a whole (i.e., 2 ideal days)
- **Story points**
  - Metric for the size of a user story, feature, or other piece of work
  - A point value to each item is assigned
  - Estimation scale:
    - *Fibonacci series*; 1, 2, 3, 5, 8, …
    - *Subsequent number as twice the number that precedes it*: 1, 2, 4, 8, …

Sprint Planning – Story Points

- Start with the most valuable user stories from the product backlog
- Take a story in that list (ideally the smallest one)
- Discuss with the team whether that estimate is accurate
- Keep going through the stories until the team have accumulated enough points to fill the Sprint

Estimation Technique – Planning Poker

– Gamified technique to estimate effort/relative size of development in Agile development

– The best way I've found for agile teams to estimate is by playing planning poker (Grenning 2002)

Agile Estimating and Planning: Planning Poker - Mike Cohn

Estimating Progress

Velocity

Estimating Progress – Velocity

- A measure of a team's rate of progress

- Sum the number of story points assigned to each user story that the team completed during the iteration

Example:

- A team completed 3 stories, 5 SPs each → velocity = 15

Estimation – of Number of Sprints

– Size estimate of the project: sum the SP estimates for all desired features

– Divide size by velocity to arrive at an estimated number of Sprits (iterations)

Estimating Velocity

- Use Historical values
- Run a Sprint (iteration)
- Make a forecast

Tracking Project Progress

Burndown chart, The Task board

Planning – Running a Sprint Using SPs, Tasks, and a Task Board

First half of the Sprint planning

- Story points and velocity to figure out what will go into the Sprint

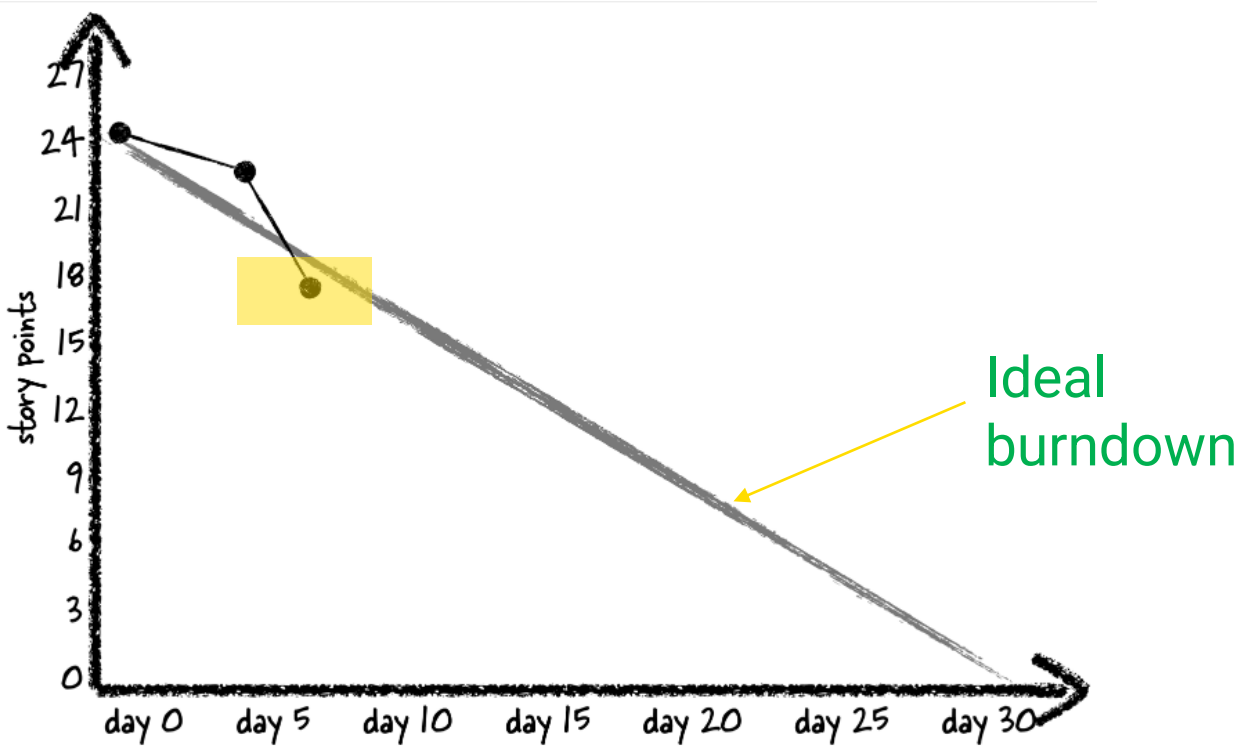
Second half of the Sprint planning

- Plan out the actual work for the team is to add cards for individual tasks
- Tasks can be written code, create design and architecture, and all those other things that teams really do every day to build and release software

Burndown Charts

- Tracks the completion of development work throughout the Sprint;
- Should be visible to everyone in the team (e.g., whiteboard, wall chart, online tool)
- First half of the Sprint planning
  - Story points and velocity to figure out what will go into the Sprint
- Good estimation and planning should help the team to burn stories relatively with similar pace

Burndown Charts based on Story Points (1)



Two stories worth 7 points burned off

Tools for Tracking Progress

Burndown chart, The Task board

Tool Support for Agile SW Development

- JIRA is a software tool for planning, tracking and managing software development
  - Supports different agile methodologies including Scrum and Kanban

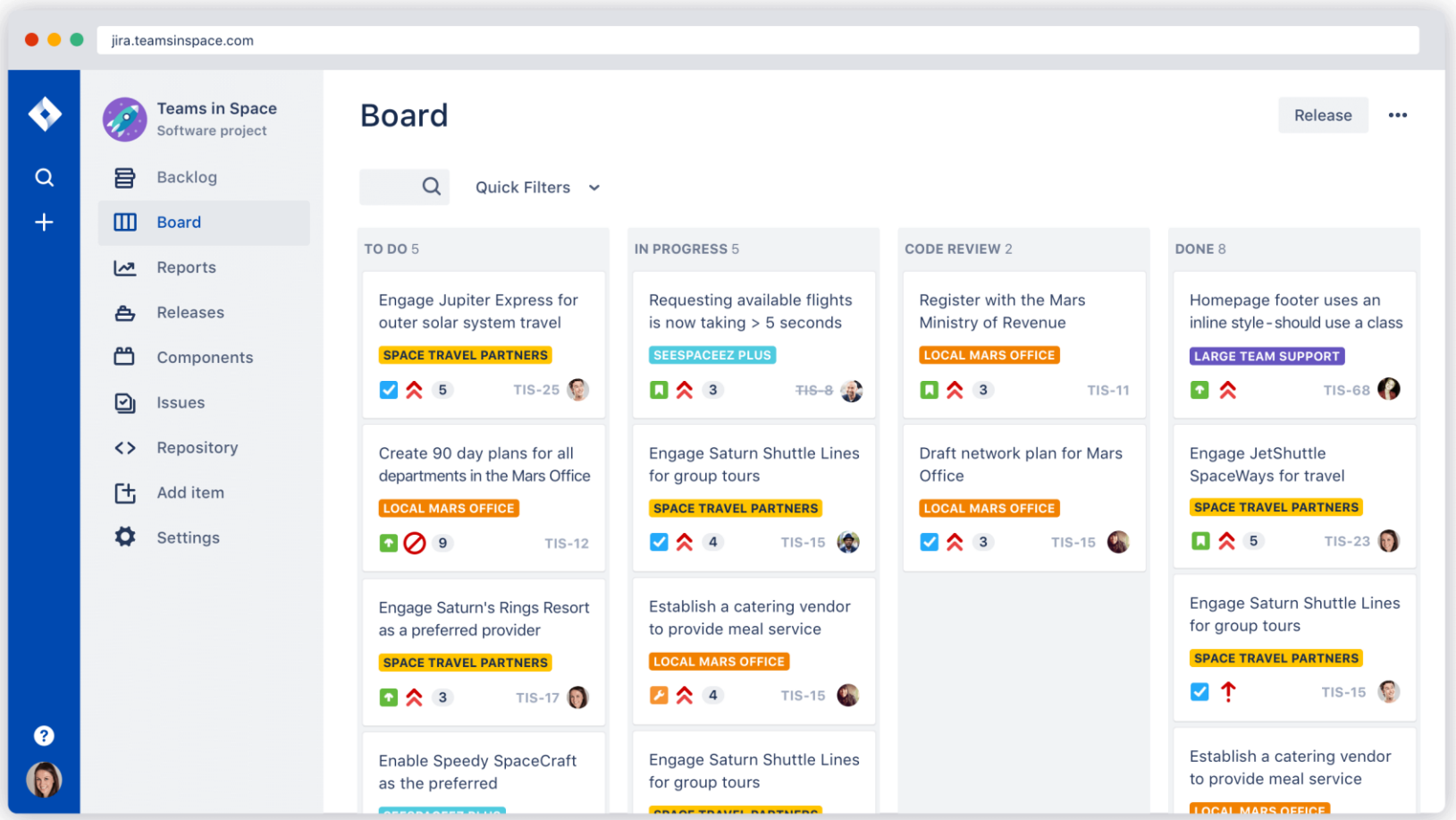
- Jira supports Scrum Sprint planning, stand ups (daily scrums), Sprints and retrospectives

- Including backlog management, project and issue tracking, agile reporting
  - E.g., Burndown and velocity charts, Sprint report

- Scrum boards visualize all the work in a given Sprint

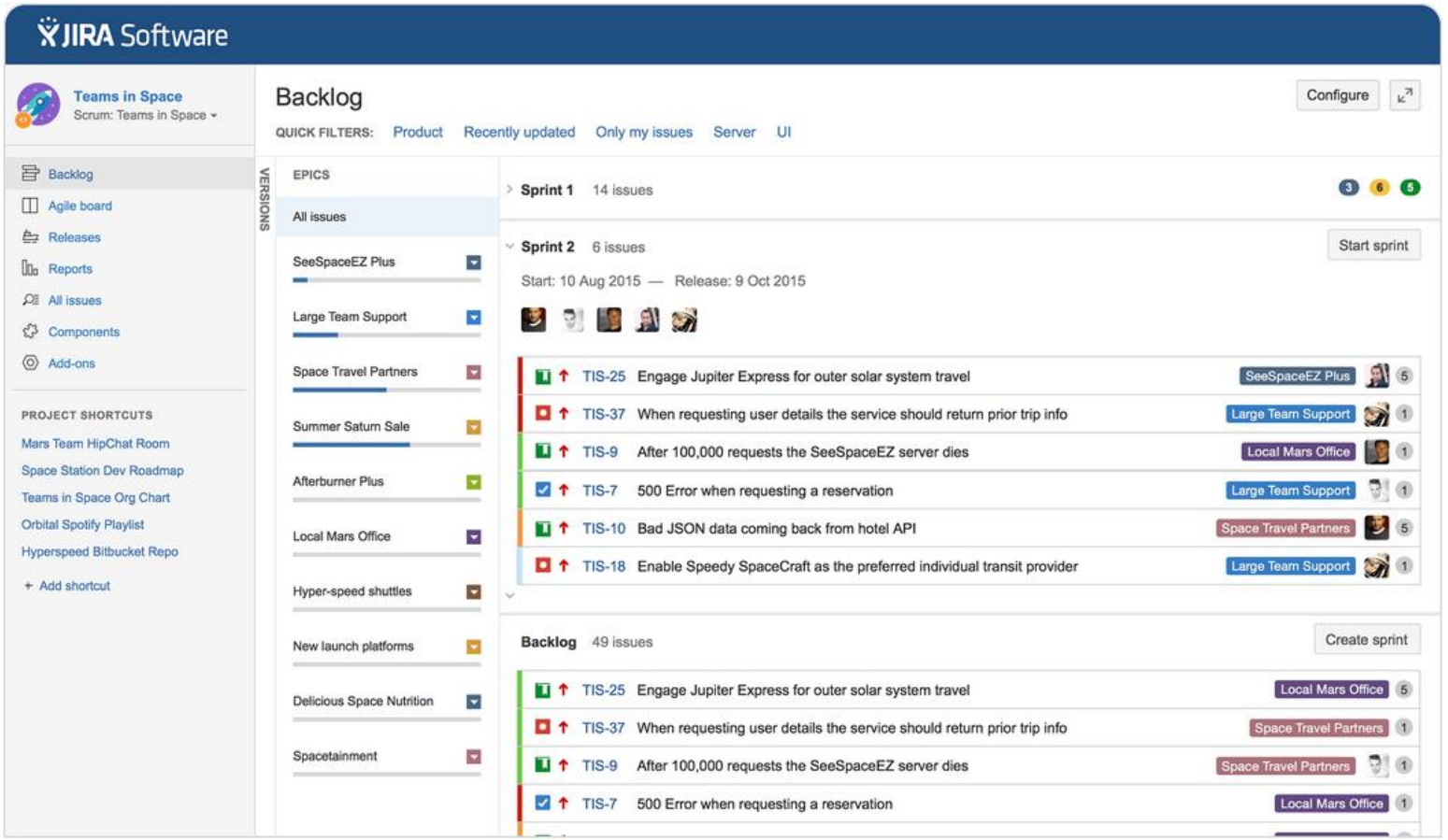
https://www.atlassian.com/software/jira/agile

Jira Agile – Scrum Board



https://www.atlassian.com/software/jira/agile

Jira Agile – Sprint Planning

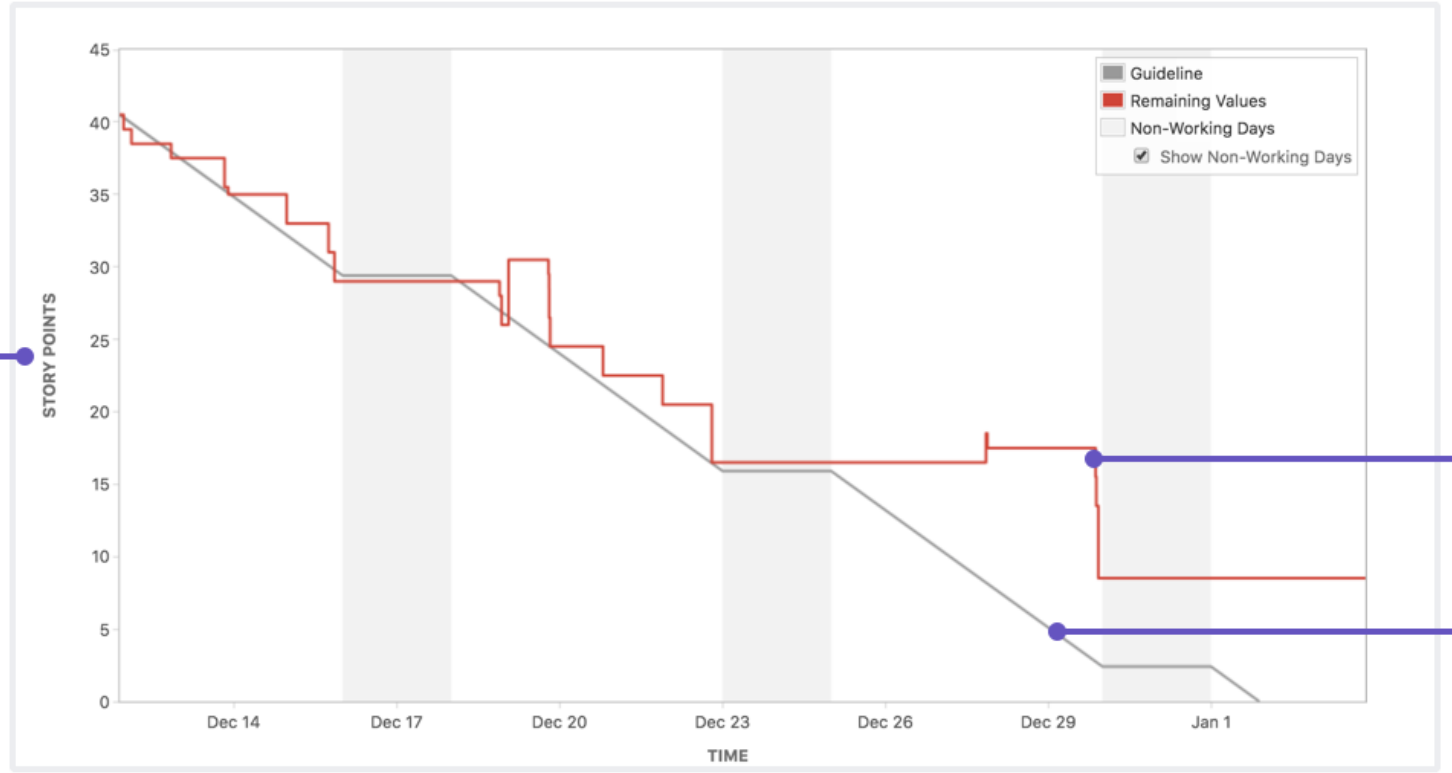


https://www.atlassian.com/software/jira/agile

Burndown Charts

- *"**Burndown**" chart* tracks the amount of estimated effort remaining in a sprint
  - Maintained daily by the Scrum Master

estimation in story points



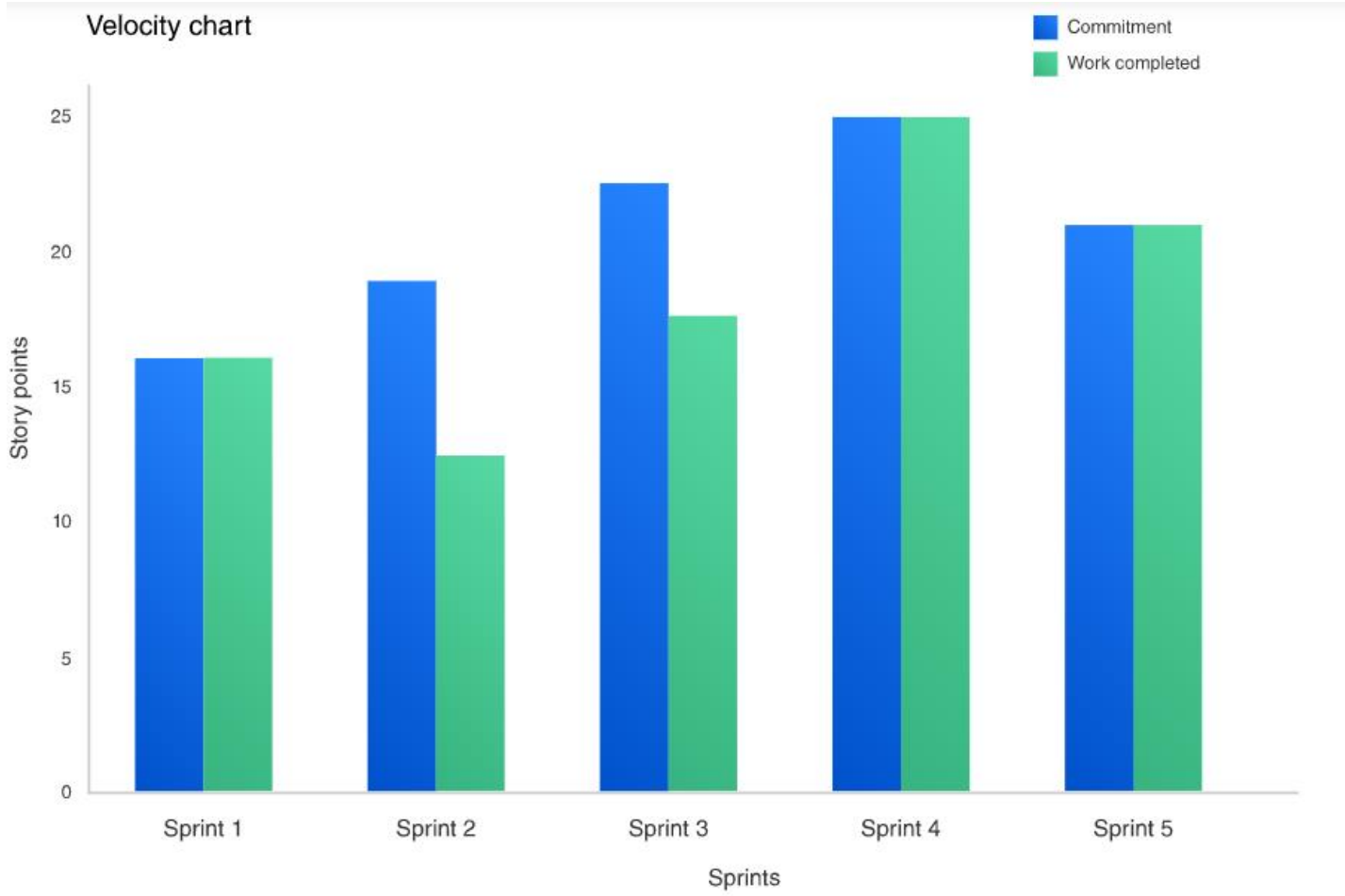
amount of work left

Guideline; ideal progress

https://www.atlassian.com/agile/tutorials/burndown-charts

Velocity Chart

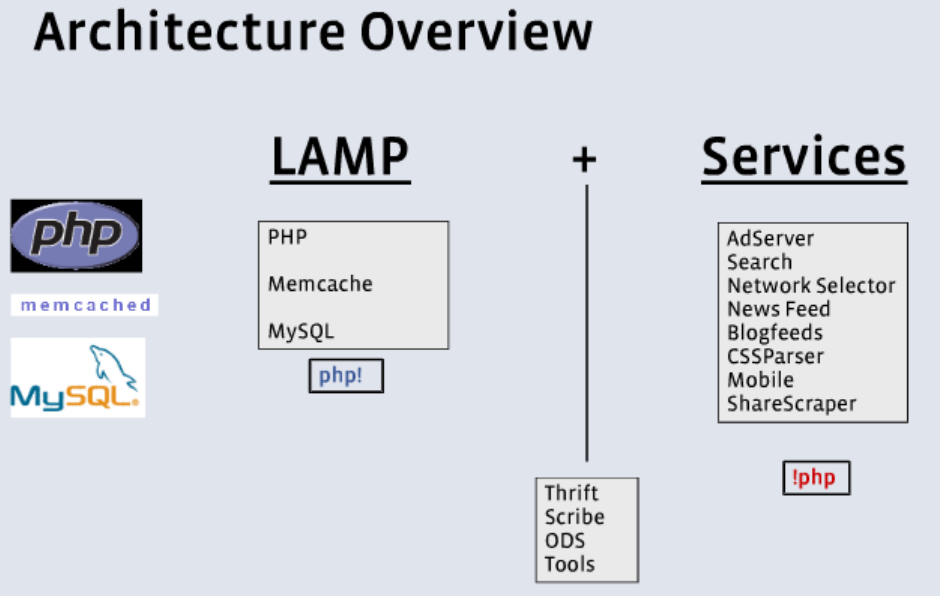
- *Velocity* tracks the amount of work from Sprint to Sprint
  Velocity chart graphically shows project/team's velocity



https://confluence.atlassian.com/jirasoftwareserver/velocity-chart-938845700.html

System/Software Architecture

Software Architecture – Technology Stack

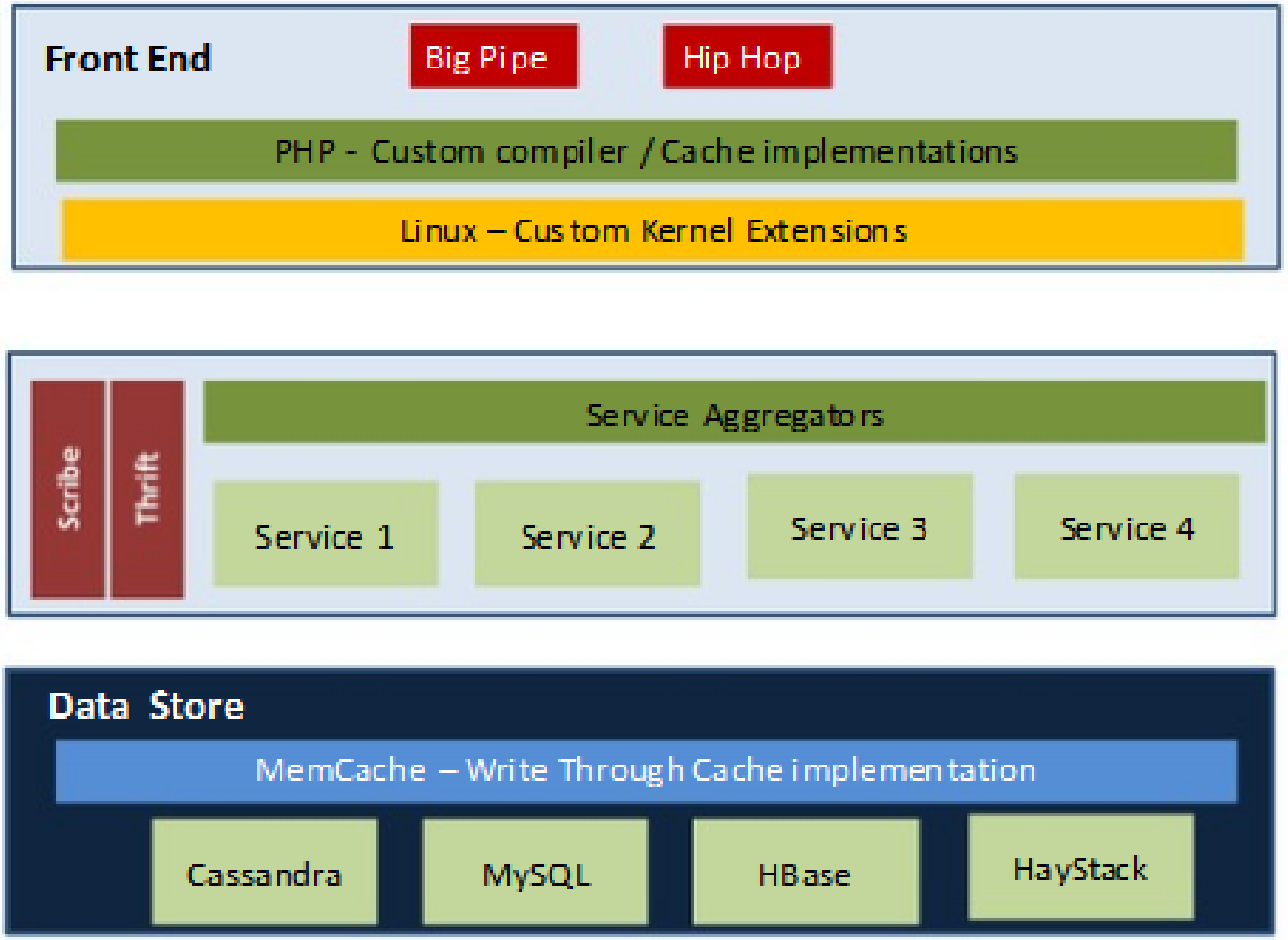


Common technology stacks:

- **WISA**: Windows, IIS Web Server, SQL server and ASP.NET
- **MEAN**: MongoDB, Express.js, Angular and Node.js

Aditya Agarwal, Facebook: Science and the Social Graph, Qcon San Francisco 2008
[http://www.infoq.com/presentations/Facebook-Software-Stack**]**

Software Architecture – Facebook



Facebook Architecture: Breaking it Open, Slide 21 [https://www.slideshare.net/AditiTechnologies/facebook-architecture-breaking-it-open]

Software Architecture – Traditional Web Application

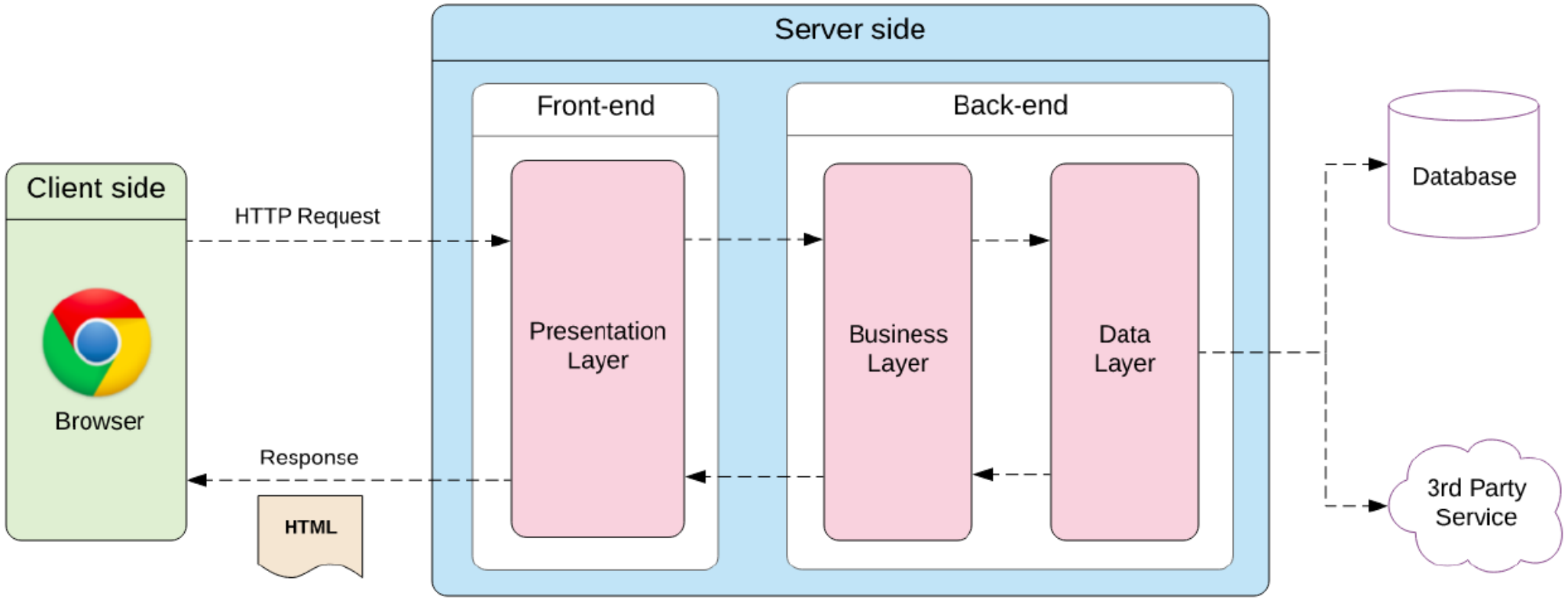
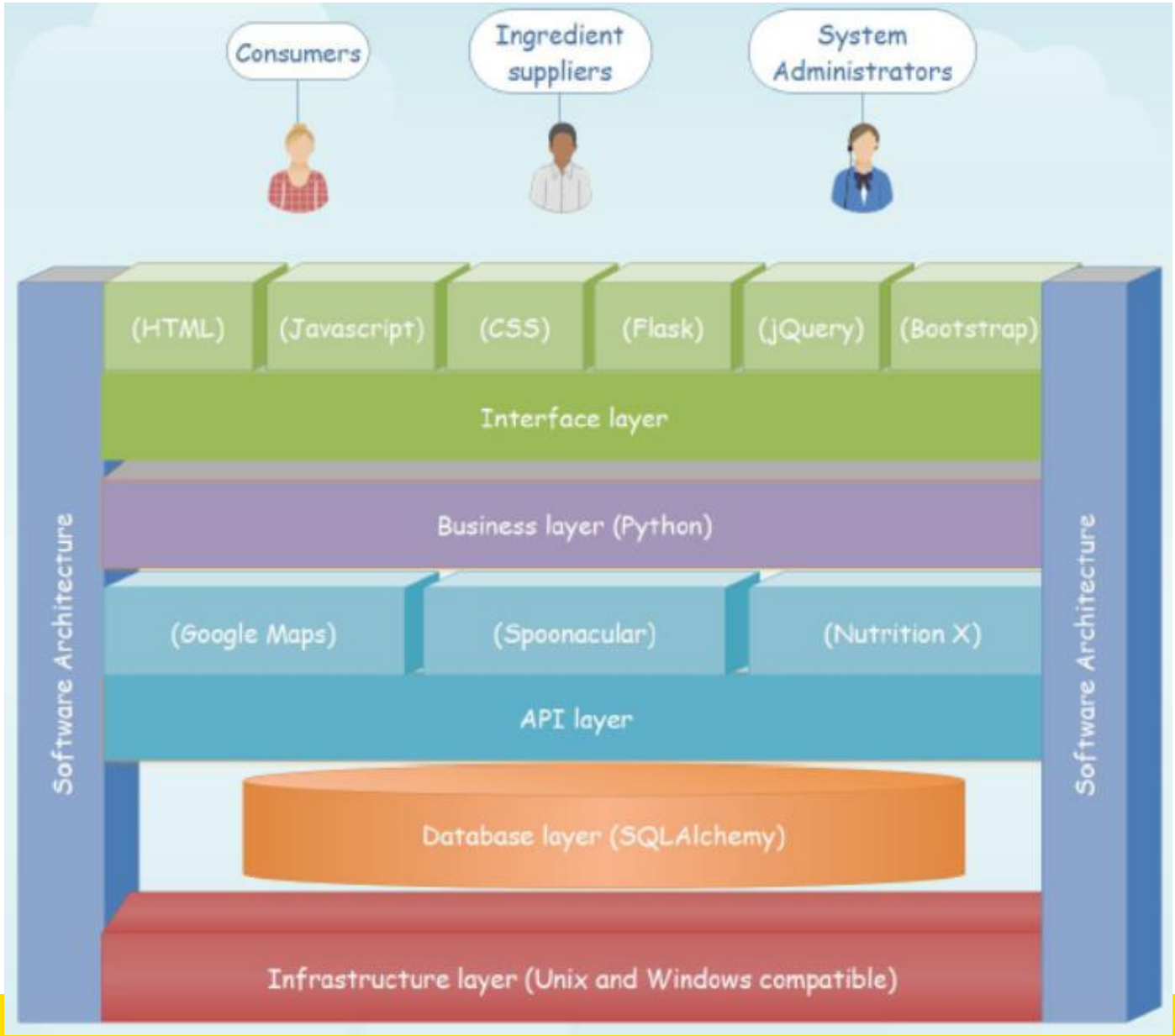


Image Source: https://elmprogramming.com/what-is-a-single-page-app.html

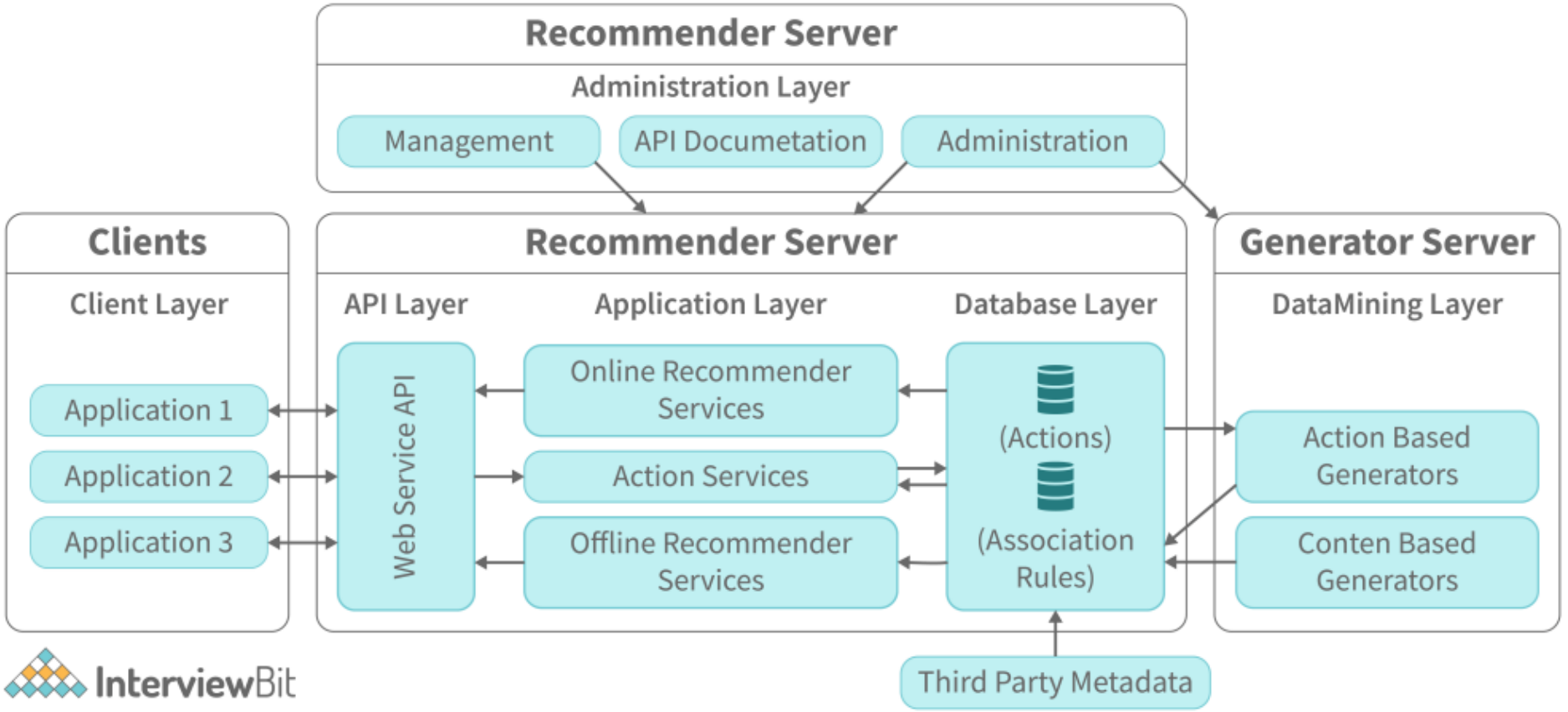
System Architecture Example

- Infrastructure layer
- Database layer
- API layer
- Business layer
- Interface layer

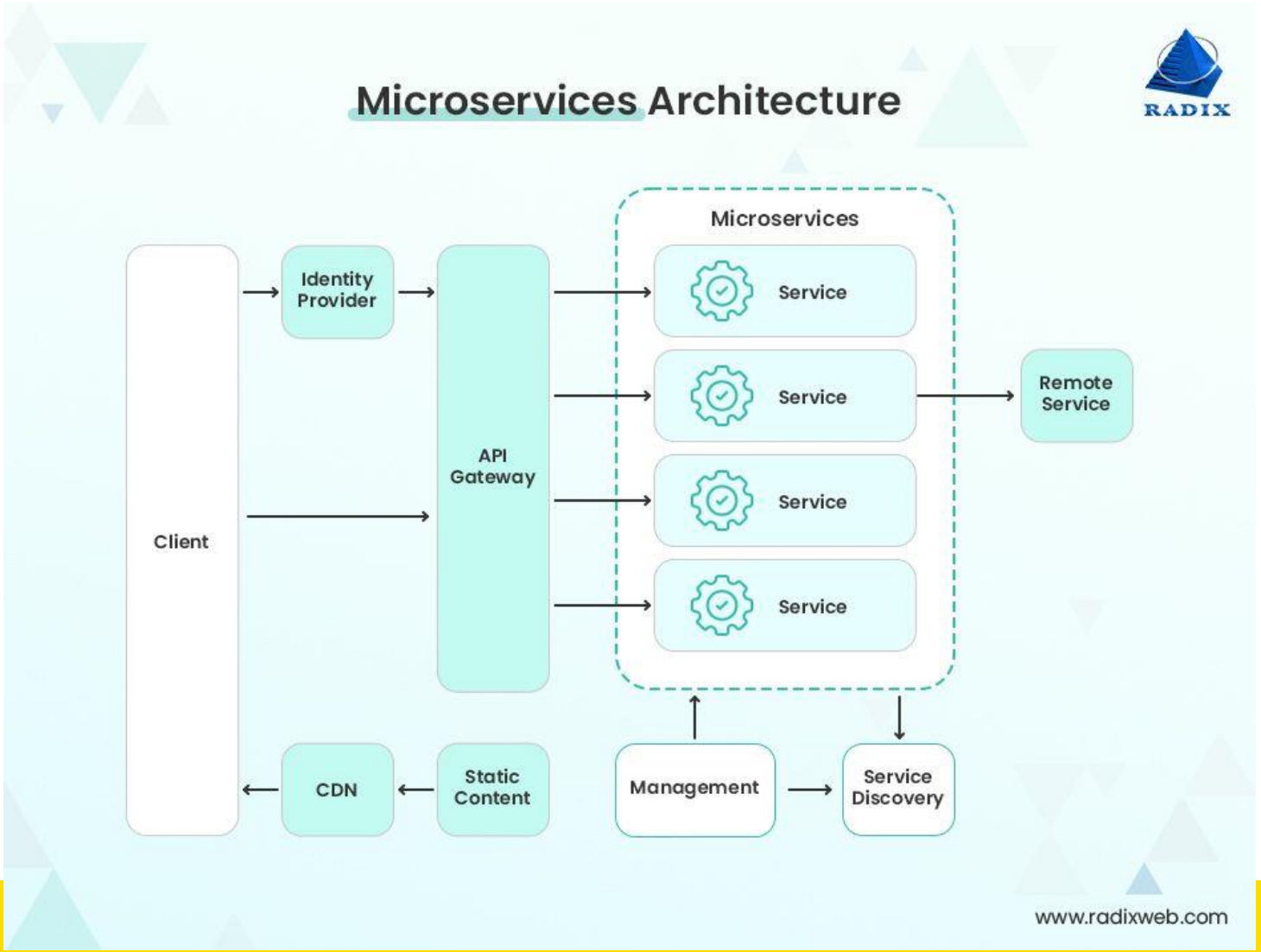


Useful Sources:
https://en.wikipedia.org/wiki/Software_architecture
https://codeburst.io/software-architecture-the-difference-between-architecture-and-design-7936abdd5830
https://www.synopsys.com/software-integrity/resources/knowledge-database/software-architecture.html

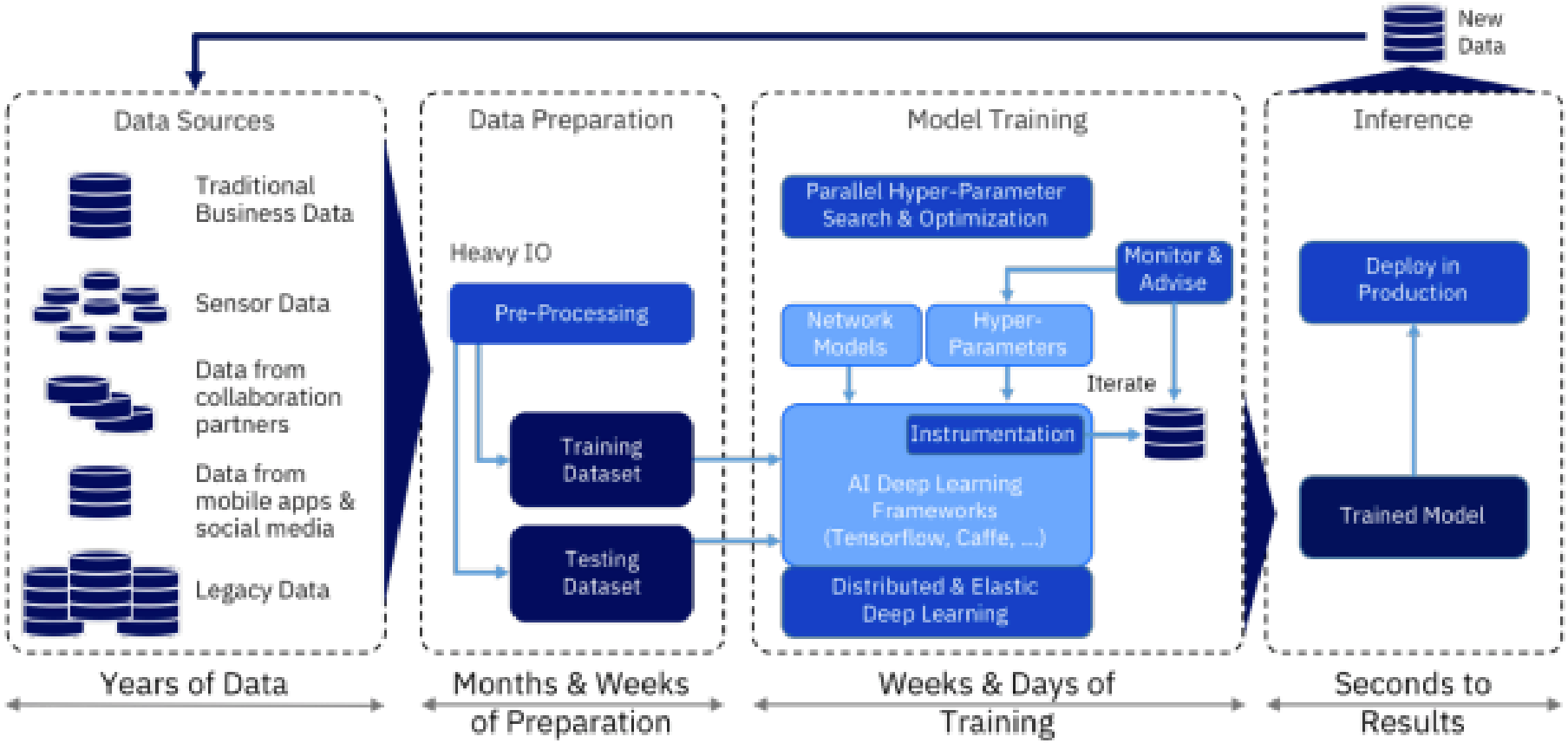
Software Architecture – Recommender System



Software Architecture – Serverless Web Application



AI-Based Software Architecture



source- https://www.ibm.com/downloads/cas/N30AY0O6

Software/System Architecture – Workshop

Location: Old Main Building G31 **Time: Thursdays 5pm - 7pm (Wk 2-5, 7-10)**

COMP[39]900 Workshops			
Week #	**Topics covered**	**Slides**	**Resources**
2	Planning and drawing your system architecture Figma and Storyboarding Intro to Primers Assessment	here	here
3	Managin client expectations Writing a good Background section & Design Justifications (useful for proposal and final report) Jira and User Stories	here	here

https://moodle.telt.unsw.edu.au/mod/page/view.php?id=7187053

Agile Project Management – Atlassian Industry Lecture

Scrum – Jira

References

- Andrew Stellman, Margaret C. L. Greene 2014. Learning Agile: Understanding Scrum, XP, Lean and Kanban (1st Edition). O'Reilly, CA, USA.

- Ken Schwaber and Jeff Sutherland, The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game, https://www.scrumguides.org/docs/scrumguide/v2017/2017-Scrum-Guide-US.pdf#zoom=100
- Agile Alliance. [agilealliance.info]
- Scrum Software Development. [https://en.wikipedia.org/wiki/Scrum_%28software_development%29]